

Lab 03: From Frozen Features to Fine-Tuning

Due date posted on Blackboard

You are given a small pretrained backbone and almost no time to train one from scratch. Fine-tune a pretrained vision backbone on an image dataset of your choosing, using the augmentation pipeline from Part 3 of the module, then compare two ways of using that backbone under a controlled, staged comparison: freezing it and training only a new head, against unfreezing part of it and fine-tuning. What is being assessed is whether you can tell what a frozen backbone's pretrained features can and cannot do for a new task, not whether you hit a particular accuracy number.

Starter code

You are given `lab03_starter.py`, containing the `train_transform` and `eval_transform` pipelines from Part 3 of the module, and a small `TinyBackbone` class with a replaceable `fc` head, structured the same way as Part 4's `resnet18` example. Do not edit this file directly; copy what you need from it into your own work.

You may use `torchvision`'s `resnet18` with pretrained ImageNet weights instead of `TinyBackbone` if you prefer and your environment can reach the internet; either backbone is acceptable, as long as you freeze and unfreeze it the way the module describes. Do not change how the backbone is structured beyond replacing its final layer; every comparison in this lab is about which parameters are allowed to move, not about redesigning the architecture.

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]),
])

eval_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]),
])

# backbone = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
# backbone.fc = nn.Linear(backbone.fc.in_features, num_classes) # replace the head
```

Run `lab03_starter.py` once, unmodified, to confirm the transform pipelines and the backbone import correctly before changing anything.

What to build

Four pieces, in order.

1. **A dataset and the two transform pipelines.** Choose a small image classification dataset, split it into training and validation sets, and build `train_transform` / `eval_transform` exactly as Part 3 describes. Confirm, the way Part 3's own "Try it" does, that five passes of `train_transform` on the same image are visibly different from each other and five passes of `eval_transform` are identical.

2. **A feature-extraction baseline.** Load a pretrained backbone, freeze every parameter, replace its final layer for your dataset's number of classes, and train only the new head. Record validation accuracy, training time, and the number of trainable parameters.
3. **A fine-tuning comparison.** Starting from the same pretrained weights, unfreeze the backbone's last block and continue training it alongside the head, using a smaller learning rate for the unfrozen backbone than for the head (discriminative learning rates, via one optimizer with multiple parameter groups). Train for the same number of epochs, on the same split and seed, as the feature-extraction run, and record the same numbers.
4. **A conclusion.** State which approach reached the higher validation accuracy on your dataset, and explain why using the mechanism behind it, not just which number is bigger. If the two approaches tie, or feature extraction wins, say so and explain why that is also a legitimate result, not a failed experiment.

Requirements

Your submission must do all of the following. Each is checked directly during marking.

- Use both transform pipelines correctly: augmentation only at training time, a fixed evaluation view at validation time. Applying `train_transform` during evaluation, or `eval_transform` during training, is treated as a pipeline error, not a modelling choice.
- Keep the frozen backbone in `eval()` mode while only the head is training, and state in your README what would happen to its BatchNorm running statistics if you left it in `train()` mode instead.
- Compare feature extraction and fine-tuning using the identical dataset split, the identical number of epochs, and the identical random seed, so the only thing that differs between the two runs is which parameters were allowed to move.
- Use discriminative learning rates for the fine-tuning run: a separate, smaller learning rate for the unfrozen backbone than for the head, expressed as parameter groups on one optimizer, not two separate optimizers.
- If augmentation, your backbone choice, or fine-tuning itself does not improve your result, report that honestly; a null result explained correctly is worth more than a flattering plot.
- Report validation accuracy, training time, and the number of trainable parameters for both runs, in a table or clearly labelled printed output.
- State your final comparison and conclusion with a mechanism-level explanation, referencing what fine-tuning changed that feature extraction could not.

Watch out

A frozen backbone still needs `eval()` behaviour for its batch normalization layers even during training of the new head: call `backbone.eval()` on the frozen portion, or, more simply, put the whole model in `train()` mode only once you actually start unfreezing and updating backbone weights. Leaving a frozen backbone in `train()` mode does not break anything outright, but its BatchNorm layers will keep updating their running statistics from your new, smaller, and differently distributed dataset, which can quietly shift the very features you are trying to keep fixed.

Why hold everything else fixed

Training the feature-extraction run for 5 epochs and the fine-tuning run for 20 would tell you fine-tuning reached a higher number, not that fine-tuning is the better approach; more epochs alone could have produced that gap. This is the same principle behind Lab 02's staged comparison: a result is only interpretable against a baseline that differs from it in exactly one place. Keep the data split, the number of epochs, the random seed, and the augmentation settings identical between your two runs, and vary only which parameters are allowed to move.

Repository requirements

The repository is as much a part of the submission as the code. It must contain the following, and it must not contain your virtual environment.

Item	Requirement
.gitignore	Present, committed before your first code commit, and excluding .venv, __pycache__, .env, and any downloaded dataset or pretrained-weights cache
README.md	Explains your dataset choice, both transform pipelines, both training runs, and how to reproduce your results
requirements.txt	Generated with pip freeze, including torch (and torchvision, if you used it)
Your code	Your data pipeline, both training runs, and the final comparison, as scripts or a single notebook
Commit history	At least three commits, each a coherent change, with messages that describe what changed

README template

You may use this structure directly. Replace the content, not the shape.

```
# From Frozen Features to Fine-Tuning

Fine-tuning a pretrained vision backbone on <your dataset>, comparing
feature extraction against fine-tuning.

## Setup
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt

## Run
python train_feature_extraction.py # freezes the backbone, trains only the head
python train_fine_tune.py         # unfreezes the last block, discriminative lr
python compare.py                 # prints the final comparison

## Dataset
Which dataset, how many classes, how many training/validation images.

## Results
Stage          val accuracy  train time  trainable params
Feature extraction: 0.XXX      X.Xs       X
Fine-tuning:      0.XXX      X.Xs       X

## Conclusion
Which approach worked better on this dataset, and why, in terms of what
fine-tuning changed that feature extraction could not.

## Known limitations
Anything you are aware of that does not work, or that you would improve
with more time.
```

How to submit

Post the URL of your GitHub repository to the Lab 03 submission on Blackboard. Do not upload files, and do not submit a link to a single file within the repository.

```
https://github.com/your-username/your-repository
```

Make sure the repository is public. If you prefer to keep it private, you must share it with the instructor account below. Sharing with any other account does not count as a submission and will be rejected.

Instructor GitHub account

`hossein.pourmodheji@humber.ca`

Add this account as a collaborator under Settings, Collaborators in your repository. Any other account will be rejected.

A repository that a marker cannot open is treated as not submitted, and the late policy applies from the point at which access is granted.

Before you submit, run this check

Clone your own repository into a different folder, create a fresh virtual environment, install from requirements.txt, and reproduce both runs following only your README. If both runs complete cleanly and agree with what you reported, your submission is complete. If it does not, you have found the problem before your marker did.